Sort Stack

AlgoExpert · Medium · Stacks

The problem

- Sort a stack (list, top = end) ascending using only push/pop/peek and recursion.

Intuition

- Recursively pop to empty, then insert each value back in sorted position.

Brute force vs optimal

- Use an auxiliary stack → $O(n^2)$
- Recursion only (no extra structures) → $O(n^2)$ time, $O(n)$ call stack
- Insertion sort on a stack

Approach

- 1. Pop the top and sort the rest
- 2. Insert it back
- 3. Insertion pops larger tops temporarily

Worked example

stack (top ↑)




```
pop everything recursively…
```

Recursively pop until empty

Worked example

stack (top ↑)




```
insert -5 into sorted stack
```

Insert -5 into position

Worked example

stack (top ↑)



insert 2 into sorted stack

Insert 2 into position

Worked example

stack (top ↑)



`insert 4 into sorted stack`

Insert 4 into position

Worked example

stack (top ↑)

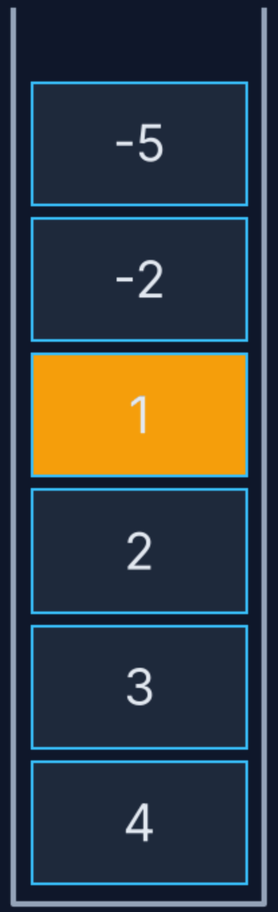



```
insert 3 into sorted stack
```

Insert 3 into position

Worked example

stack (top ↑)




```
insert 1 into sorted stack
```

Insert 1 into position

Java solution

```java
public class SortStack {
    public static ArrayList<Integer> sortStack(ArrayList<Integer> stack) {
        if (stack.isEmpty()) return stack;
        int top = stack.remove(stack.size() - 1);
        sortStack(stack);
        insert(stack, top);
        return stack;
    }

    private static void insert(ArrayList<Integer> st, int v) {
        if (st.isEmpty() || st.get(st.size() - 1) <= v) { st.add(v); return; }
        int top = st.remove(st.size() - 1);
        insert(st, v);
        st.add(top);
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n) recursion

Edge cases & pitfalls

- Base case: empty stack
- Insert pops while top > value
- Recursion depth = n

Interview tips & follow-ups

- Mention the auxiliary-stack iterative version

Recap

- Recursively pop to empty, then insert each value back in sorted position.
- Complexity: O(n^2)